

Documentación Técnica — Evals de Aplicaciones con IA

Documentación de referencia del diseño, las decisiones técnicas y el funcionamiento del proyecto.

Contenido

1. Por qué el testing de IA es distinto
2. El sistema bajo prueba
3. Golden dataset
4. Los scorers
5. LLM-as-judge y sus riesgos
6. Umbrales sobre métricas continuas
7. Regresión de prompts
8. Mock determinista vs proveedor real
9. Manejo del no determinismo
10. Integración en CI
11. Vías de extensión
12. Glosario

1. Por qué el testing de IA es distinto

El testing tradicional se basa en el **determinismo**: `assertEquals(suma(2,3), 5)` da el mismo resultado siempre. Una aplicación basada en un LLM rompe ese supuesto en tres frentes:

- **No determinismo**: la misma entrada puede producir salidas distintas entre corridas.
- **Salida abierta**: para muchas tareas (resumir, redactar) no existe una única respuesta correcta contra la cual comparar.
- **Degradación silenciosa**: un cambio de prompt o una actualización del modelo puede empeorar la calidad sin romper ningún test tradicional.

Por eso el testing de IA pasa de **verificar igualdad exacta** a **evaluar propiedades y distribuciones** sobre un conjunto de casos. Ese es el rol de las **evals**.

2. El sistema bajo prueba

Para que el harness sea concreto, el sistema bajo prueba es una aplicación con IA real y acotada: un **clasificador de tickets de soporte** que asigna cada mensaje a una categoría (`billing` , `technical` , `account` , `other`).

Su lógica es representativa de una aplicación LLM típica: la inteligencia vive en el **prompt** (`buildPrompt`) y en el **post-procesamiento** de una salida abierta (`parseCategory`). El clasificador no depende de un proveedor concreto, sino de una interfaz `LlmClient` .

3. Golden dataset

El golden dataset (`evals/dataset.json`) es el conjunto de casos con su respuesta esperada — el equivalente a la suite de regresión del software tradicional. Cada ítem tiene un `input` (el mensaje) y un `expected` (la categoría correcta).

Un buen dataset se construye con: casos reales tomados de producción, casos de borde diseñados a mano, y **cada nueva falla que aparece se agrega como caso**. Así el dataset evoluciona con la realidad y una misma falla no vuelve a pasar sin ser detectada. Este proyecto incluye un caso deliberado de **señales mixtas** ("La factura mensual no se descarga") para mostrar que el dataset debe capturar los casos difíciles, no solo los fáciles.

4. Los scorers

Testear IA no es comparar por igualdad: se evalúan propiedades con **scorers**. El proyecto implementa tres, del más estricto al más flexible:

Exact match

Para tareas con una respuesta única correcta (como la clasificación). Devuelve 0 o 1. Es el scorer que usa el gate del clasificador.

Similitud semántica

Para salidas abiertas (resúmenes, respuestas) donde no hay una única forma correcta: dos textos pueden decir lo mismo con distintas palabras. Devuelve 0..1. Acá se implementa como **coseno sobre bolsa de palabras** — una aproximación léxica **determinista y offline**. En producción se usarían **embeddings** reales (vectores semánticos de un modelo), pero la aproximación léxica alcanza para demostrar el concepto y correr sin costo.

LLM-as-judge

Para calidad subjetiva (tono, completitud, adherencia a una política) donde ni el exact match ni la similitud alcanzan. Un modelo, con una rúbrica, puntúa la respuesta de otro. Acá se implementa como una **heurística determinista** que emula al juez (verifica términos requeridos, prohibidos y longitud), para poder testearla sin costo ni no determinismo.

5. LLM-as-judge y sus riesgos

Usar un LLM para evaluar a otro permite medir calidad subjetiva **a escala**, algo imposible de hacer solo con métricas simples o con revisión humana de miles de casos. Pero tiene riesgos conocidos que un QA de IA debe conocer:

- **Sesgo de posición:** favorece la primera opción en comparaciones.
- **Preferencia por respuestas largas** aunque no sean mejores.
- **Auto-preferencia:** tiende a preferir salidas de su misma familia de modelos.
- **Inconsistencia** entre corridas.

Mitigaciones: una rúbrica **específica y con ejemplos** (no un "evalúa del 1 al 10" vago); usar como juez un modelo **distinto** del evaluado; **calibrar** periódicamente el juez contra evaluación humana en una muestra; y hacer spot-checks. El juez es una herramienta poderosa, pero hay que auditarlo, no confiar ciegamente.

6. Umbrales sobre métricas continuas

Con métricas continuas no se exige 100 %. Se fija un **umbral empírico** según la criticidad de la dimensión: estricto en seguridad y factualidad, más laxo en estilo. El proceso correcto:

1. **Medir el baseline** del sistema actual (no inventar un umbral en el aire).
2. Fijar el umbral sobre esa evidencia (ej: " ≥ 80 % de exactitud").
3. Gatear también la **regresión relativa**: una caída brusca contra el baseline bloquea aunque el valor absoluto siga sobre el umbral.

En este proyecto, el umbral (`ACCURACY_THRESHOLD = 0.8`) actúa como gate: el runner termina con código distinto de cero si la exactitud cae por debajo, y el pipeline falla. La exactitud medida (92,3 %) supera el umbral con margen, dejando espacio para la variabilidad real de un modelo.

7. Regresión de prompts

En una aplicación LLM, el **prompt es código** y debe versionarse. Un error frecuente es ajustar el prompt para mejorar un caso puntual y, sin saberlo, degradar otros. Eso es una **regresión de prompt**.

La suite de evals es la defensa: cada cambio de prompt se corre contra el golden dataset y se compara con el baseline; solo se aprueba si mejora el caso objetivo **sin degradar el resto**. Sin esta suite, cada "mejora" de prompt es una apuesta a ciegas sobre qué otros casos se rompieron. Es el mismo rol que la suite de regresión cumple en el software tradicional, aplicado a la parte no determinista del sistema.

8. Mock determinista vs proveedor real

El sistema depende de la interfaz `LlmClient`, con dos implementaciones:

- `MockLlmClient` — determinista, basado en reglas. Se usa en el CI: **gratis, reproducible y sin latencia**. Comete algunos errores a propósito (como un modelo real), pero siempre los mismos.
- `AnthropicLlmClient` — el proveedor real (Claude, vía el SDK de Anthropic). Es el camino a producción; requiere `ANTHROPIC_API_KEY`. Usa el modelo `claude-opus-4-8` por defecto.

Por qué el gate usa el mock: correr un modelo real en cada pull request sería caro (cuesta por llamada), lento (latencia de red) y no determinista (el gate flaquearía). El mock da un gate confiable y gratuito; el proveedor real se reserva para corridas de evaluación contra producción, donde interesa medir el modelo real. Como ambos cumplen el mismo contrato, intercambiarlos no toca ni el clasificador ni las evals.

9. Manejo del no determinismo

Un LLM puede responder la misma categoría de muchas formas: `"billing"`, `"Billing."`, `"La categoría es billing"`, `" TECHNICAL\n"`. El sistema debe ser **robusto a esa variabilidad de formato**. La función `parseCategory` normaliza la salida buscando la primera categoría conocida mencionada, y los tests verifican explícitamente que tolera todos esos formatos.

Este post-procesamiento tolerante es una parte esencial (y a menudo olvidada) de construir aplicaciones LLM confiables: no alcanza con un buen prompt, hay que blindar el parseo de la salida.

10. Integración en CI

El pipeline corre en cada push/PR: type-check, los tests unitarios (scorers, clasificador, suite de evals) y el **gate de evals** (`npm run evals`). Todo con el modelo mock, así que es determinista, gratuito y no requiere API key. Si un cambio hace caer la exactitud por debajo del umbral, el gate falla y bloquea el merge.

Para controlar costo y latencia cuando se evalúa con un modelo real, la estrategia es por capas (como en cualquier pipeline): un subconjunto barato del dataset en cada PR, y la evaluación completa contra el modelo real en una corrida programada o pre-release.

11. Vías de extensión

- **Embeddings reales:** reemplazar la similitud léxica por embeddings de un modelo para una similitud semántica genuina.
- **LLM-as-judge real:** conectar el juez a un modelo real con una rúbrica y calibrarlo contra evaluación humana.
- **Testing de RAG:** evaluar por separado la recuperación (precision/recall) y la generación (groundedness).
- **Testing de agentes:** evaluar la trayectoria (qué herramientas llamó, en qué orden), no solo el resultado final.
- **Red teaming:** una suite adversarial (prompt injection, jailbreaks) como métrica de regresión de seguridad.
- **Herramientas de la industria:** integrar promptfoo, Langfuse, Ragas o DeepEval para tracing y evals sobre datos de producción.

12. Glosario

- **Eval:** evaluación sistemática de la calidad de un sistema con IA sobre un conjunto de casos.
- **Golden dataset:** conjunto de casos con su respuesta esperada; la base de la evaluación.
- **Scorer:** función que puntúa una salida (exact match, similitud, juez).
- **Exact match:** coincidencia exacta; para respuestas únicas.
- **Similitud semántica:** cercanía de significado entre dos textos.
- **Embeddings:** representación vectorial del significado de un texto.
- **LLM-as-judge:** usar un modelo, con una rúbrica, para puntuar la salida de otro.
- **Umbral:** valor mínimo aceptable de una métrica; actúa como gate.
- **Regresión de prompt:** cambio de prompt que degrada la calidad global.
- **Groundedness:** grado en que una respuesta se sostiene en las fuentes provistas (relevante en RAG).
- **No determinismo:** que la misma entrada pueda producir salidas distintas.